VITYARTHI

OPEN SOURCE SOFTWARE - NGMC

WINTER SEMESTER - 2026

CASE STUDY

(FACULTY NAME: DR. VIKAS PANTHI)

(SLOT : E22)

~ SUBMISSION BY ~

TEJAS SANTOSH PAITHANKAR

24BCY10104

CASE STUDY ON - LIBRE OFFICE

Introduction

Open source software is often understood as a model built on freedom, collaboration, and,, shared ownership. However, the reality behind many large open source projects is more complex. This case study examines the events that led to the creation of LibreOffice, not just as a technical fork, but as a response to deeper issues of governance, trust, and,, control.

By, analysing this case, the report aims to explore the ethical boundaries of corporate involvement, in open source and, evaluate whether true openness can exist without shared decision-making power.

Section A: Understand,ing the Context Behind the LibreOffice Fork

At first glance, the story of LibreOffice, looks like just another software project. But when you look a little deeper, it becomes clear that it is really about control, trust, and, how “open” open source actually . OpenOffice.org, for a long time, appeared to be a successful example of open source software. It was free, widely used, and, supported by a large global community. Many people see it as a strong alternative to proprietary tools like Microsoft Office. On the surface, everything seems to be working well. However, there was a deeper issue that was not immediately visible. The problem was not with the software itself, but with how it was governed.

Even though OpenOffice.org was open source, the control of the project was not fully in the hand,s of the community. Contributors were required to sign a copyright assignment agreement with Sun Microsystems. In simple terms, this meant that while developers were writing and, improving the code, the company still held shared ownership over it. This created a kind of imbalance. The community was actively contributing, but it did not have equal authority in decision-making.

A simple way to understand, this is by thinking of a WhatsApp group. Everyone in the group can participate, share ideas, and, stay active. But at the end of the day, the admin has the final say what stays, what gets removed, and, even who remains in the group. In a

similar way, contributors in OpenOffice.org could suggest and, build, but the final decisions were controlled by Sun. At first, this arrangement was tolerated. It might not have felt ideal, but it worked because Sun was seen as relatively supportive of the open source community. So even if there was some imbalance, there was still a level of trust. This is exactly why the situation changed so quickly when Oracle acquired Sun Microsystems in 2010. The acquisition did not introduce a completely new problem, but it made the existing one much more serious. Oracle had a reputation for being more commercially driven and, less community-focused. Because of this, contributors started to look at the same governance structure differently. What was earlier uncomfortable now started to feel risky.

Oracle continued with similar practices which including the copyright model but communication became weaker and, decisions felt more centralized. It also introduced commercial elements, such as a paid version of OpenOffice. This created the impression that community contributions were being used for corporate benefit, without meaningful involvement from the contributors themselves.

At this point, the issue was no longer just structural and, it became a matter of trust. Contributors began to feel that the project no longer reflected the values of open source. The earlier acceptance of copyright assignment turned into a serious concern, especially under a company known for strict control over intellectual property. In a way, nothing drastic had changed in policy, but the perception had completely shifted. This situation highlights an important distinction between legal openness and, practical openness. Legally, OpenOffice.org was still open source. Anyone could access the code, modify it, and, distribute it. But in practice, the direction of the project was controlled by a single entity. So while freedom existed on paper, it was limited when it came to actual decision-making.

This is where the core issue lies. Open source is not just about access to code it is also about who gets to decide how that code evolves. The LibreOffice fork did not happen suddenly. It was the result of a growing imbalance between contribution and, control, combined with a breakdown of trust after Oracle's acquisition. Over time, it became clear that without shared governance, the system was not sustainable. In the end, this case shows that open source is not just a technical concept. It is also a social system built on trust, participation, and, fair distribution of power. When any of these elements weaken, even a successful project can reach a breaking point.

Section B: The Ethics of Corporate Control

1. Does Investment Justify Control?

One of the strongest arguments in favor of corporate control is quite straightforward: companies invest a lot into open source projects. They provide funding, maintain infrastructure, pay developers, and, hand,le legal complexities. Looking at it this way, it does seem reasonable for them to expect some level of control in return. In fact, without this kind of support, many large open source projects might not even survive. OpenOffice itself benefited from Sun Microsystems' backing in its early years. So it would be unfair to ignore the role companies play in keeping such projects running.

However, the issue starts to appear when control becomes too concentrated. While companies provide resources, a significant part of the actual development often comes from the community. When one entity continues to hold decision-making power despite collective contributions, it creates a noticeable imbalance. In the case of OpenOffice, the problem was not corporate involvement itself. It was the extent of control. This leads to a deeper question: does financial investment automatically give a company the right to dominate something that is built collaboratively?

2. Open Source as a Shared Commons

Another way to look at open source is to see it as a shared space — almost like a digital commons. Most contributors are not there for direct financial gain. They participate because they want to learn, collaborate, or build something meaningful. In such a system, there is an expectation that no single entity should have complete control over the collective effort. From this perspective, excessive corporate control can feel like a form of appropriation. The community contributes time and, effort, but the authority remains concentrated within the organization. Over time, this creates a gap between who builds the project and, who controls it.

At the same time, it would not be realistic to ignore the importance of companies. Open source projects still need coordination, infrastructure, and, long-term direction — things that corporations are often better equipped to hand,le. So, there is a clear tension here. On one side, open source functions as a collaborative commons. On the other, it is treated as a strategic asset. The challenge is finding a balance between the two.

3. The Ethics of Copyright Assignment

The copyright assignment requirement in OpenOffice brings out one of the most uncomfortable parts of this case. Under Sun Microsystems, contributors had to share ownership of their code. Even though this was not ideal, many people accepted it because Sun was seen as relatively trustworthy. So the system continued, even if it was not perfect. But things changed after Oracle took over. The agreement itself did not change much, but its meaning did. Now, contributors were effectively giving rights to a company that was seen as more aggressive and, profit-oriented. Naturally, this made people uneasy. This raises an important question: is it fair to ask volunteers to give up ownership of their work?

On one side, such agreements can make legal processes easier. They allow companies to manage licensing and, protect the project more efficiently. But on the other side, they shift power away from contributors and, place it in the hand,s of a single entity. What makes this issue more complex is that it depends heavily on trust. The same agreement can feel acceptable under one organization and, deeply problematic under another. So it is not just a legal matter it is also a question of confidence in those who hold that power.

4. Stewardship vs Ownership

A useful way to understand, this situation is to look at the difference between ownership and, stewardship. Ownership is about control. It means having the authority to make decisions and, shape the project based on one's own interests. Stewardship, however, is more about responsibility. It involves guiding the project while still respecting the people who contribute to it. Ideally, open source projects should lean more towards stewardship than ownership. Companies can support the project, provide resources, and, even help with direction, but they should not override the collective voice of the community. The OpenOffice case shows what happens when this balance is not maintained. Even though the project was technically open, its governance started to feel controlled. In a way, it began to resemble a proprietary system, despite being open source. A more balanced approach would require transparency, shared decision-making, and, less dependence on centralized authority.

5. A Balanced Position

Looking at all these arguments together, it becomes clear that corporate involvement in open source is not inherently negative. In many cases, it is necessary for growth and, long-term sustainability. At the same time, control without accountability creates problems. Open source depends heavily on trust, and, once that trust starts to weaken, the

entire system becomes unstable. In OpenOffice, the governance structure allowed a corporation to maintain control over a project that depended significantly on community effort. This imbalance became much more visible after Oracle's acquisition.

So, a balanced view would be that companies do have a role in open source projects, but that role should come with limits. Control needs to be shared, decisions should be transparent, and, contributors should feel that their participation actually matters. Without this balance, a project risks losing not just its contributors, but the very foundation it is built on.

Section C: The Forking Question

1. Was Forking the Right Decision?

The decision to fork OpenOffice and, create LibreOffice was definitely not a simple one. Forking a major project comes with real risks. It can split the community, duplicate efforts, and, leave users confused about which direction to follow. In the short term, it can even slow things down. At first, it might seem like the better option was to stay within Oracle's system and, try to improve things from the inside. That way, the community could have continued using the existing infrastructure, brand, recognition, and, user base that OpenOffice had already built over the years. But this way of thinking depends on one key assumption that meaningful change was actually possible within Oracle's model. From what we see in the case, that did not seem very likely. Decision-making was centralized, communication was limited, and, contributors felt that their role was becoming less significant over time. If this had continued, it could have led to frustration and, eventually reduced participation. Seen from this angle, the fork starts to make more sense. It was not just a technical move, but a strategic decision. It gave the community a chance to take back control, even if it meant giving up short-term stability.

2. Oracle's Decision: Business vs Ethics

When The Document Foundation invited Oracle to participate in the new governance structure, Oracle chose not to join. This decision looks very different depending on how you view it. From a business perspective, the decision is understand,able. OpenOffice was a valuable asset, and, giving up control would have meant losing potential commercial advantages. Companies usually act in ways that protect their strategic interests, so in that sense, Oracle's response was not surprising.

However, from an ethical point of view, the situation feels different. By refusing to engage with a more open and, community-driven model, Oracle reinforced the idea that it prioritized control over collaboration. This decision further weakened the already fragile trust between the company and, the contributors. So, there is a clear gap here. What makes sense from a business stand,point does not always align with what feels fair or responsible in a community-driven environment.

3. The Cost of Forking

Forking OpenOffice did not come without consequences.
The most immediate impact was the division within the community. Contributors had to choose between staying with OpenOffice or moving to LibreOffice. Naturally, this created uncertainty and, disrupted the existing flow of development.

Users were also affected. Many organizations were unsure whether to switch to LibreOffice or continue with OpenOffice. This confusion could have reduced confidence in both projects, at least in the beginning. There was also the challenge of rebuilding. LibreOffice had to establish its own identity, governance system, and, coordination mechanisms. All of this required time and, effort. Even with these challenges, the long-term outcome tells a different story. LibreOffice gradually gained stronger support and, became more actively developed. So while the short-term effects were disruptive, the long-term results suggest that the effort was not wasted.

4. The Strongest Argument Against Forking

A strong argument against forking is that the community should have stayed and, tried to bring change from within. By remaining part of OpenOffice, contributors could have continued benefiting from its existing ecosystem. There is also a possibility that, over time, consistent pressure from the community might have pushed Oracle towards more transparency and, inclusion. Another concern is that forking divides resources. Instead of focusing efforts in one place, development gets split, which can slow down progress and, reduce overall impact. This line of reasoning suggests that forking should ideally be a last option, used only when all other possibilities have been exhausted.

5. Final Evaluation: Was the Fork Worth It?

Even though the argument against forking is quite strong, it depends heavily on the idea that the system was open to change. In the case of OpenOffice, there was very little

indication that Oracle was willing to adjust its governance approach or share control. Without that willingness, staying within the system might not have solved the core issue. Also, without trust, collaboration becomes difficult to sustain. Even if contributors had stayed, the underlying problems would likely have continued in the background. Over time, this could have resulted in a slow decline rather than an immediate split. Because of this, forking can be seen as a difficult but necessary step. It allowed the community to prioritize long-term sustainability and, control over short-term convenience. Looking back, the success of LibreOffice supports this decision. It shows that sticking to principles like openness and, shared governance can lead to stronger outcomes, even if the path is more challenging.

Section D: Lessons and, Generalisation

1. The Pattern: LibreOffice and, MariaDB

The LibreOffice case does not stand, alone. Around the same period, something very similar happened with MySQL, another major open source project that also came under Oracle's control after the acquisition of Sun Microsystems. The original creator of MySQL decided to fork the project and, created MariaDB, which followed a more community-driven approach. Over time, MariaDB gained strong adoption, especially across Linux systems. When we look at both cases together, a pattern starts to appear. Two large and, successful open source projects faced similar issues under the same corporate ownership, and, both resulted in forks. This suggests that the problem was not limited to a single project. Instead, it points to a deeper issue when corporate control becomes too dominant, it can disturb the balance between organizational interests and, community trust.

In a way, this highlights something important. Open source projects are not just built on code and, licenses. They also depend heavily on the relationship between contributors and, those in control. Once that relationship weakens, fragmentation becomes much more likely.

2. A Modern Hypothetical: Company X

Consider a hypothetical scenario where a large technology company, referred to as Company X, contributes most of the code to an open source project, controls its governance, and, uses the project's success to support its commercial services. At first glance, this may seem different from the OpenOffice situation because Company X is

actively contributing and, not just controlling. However, the core issue remains similar — centralized authority. The key question is not just who contributes the most, but who makes the decisions. If Company X controls governance without transparency or accountability, the project may still face the same risks as OpenOffice. Contributors might feel that their participation does not translate into meaningful influence. To determine whether such a corporate steward can be trusted, certain criteria become important:

- Transparency in decision-making
- Shared governance structures
- Freedom for contributors to participate meaningfully
- Absence of restrictive legal conditions like forced copyright transfer

If these conditions are not met, the situation may not be very different from the Oracle case, even if the surface-level involvement appears more positive.

3. Governance Recommendations for Future Projects

If a group of developers were starting a new open source project today, the LibreOffice case provides several important lessons for designing a sustainable governance structure.

First, copyright ownership should remain with individual contributors rather than being transferred to a single entity. This ensures that no organization gains excessive legal control over community contributions.

Second, governance should be distributed rather than centralized. Decision-making authority can be placed in the hand,s of an elected group that represents both contributors and, stakeholders, rather than a single company.

Third, transparency should be a core principle. Major decisions related to development, releases, and, project direction should be openly discussed and, documented.

Fourth, corporate participation should be encouraged but limited in influence. Companies can provide resources and, developers, but safeguards should exist to prevent them from dominating governance.

Finally, there should always be a clear and, accessible process for community feedback and, conflict resolution. This helps maintain trust and, prevents issues from escalating into major crises.

4. Broader Lesson

The biggest takeaway from this case is that open source is not just about access to code. It also depends on governance, trust, and, how power is shared among participants. These elements are often less visible, but they play a crucial role in the long-term success of a project. Without them, even a technically open system can start to feel controlled. and, once that happens, the idea of community-driven development begins to lose its meaning.

Section E: Reflection

Before reading this case study, my understand,ing of open source software was quite basic. I mostly saw it as software that is free to use and, available to everyone. I did not really think about who controls it or how decisions are actually made behind the scenes. After going through this case, my perspective has changed quite a bit. I now realize that open source is not just about access to code, but also about governance and, control. A project can appear open in terms of availability, but still be quite restricted when it comes to decision-making. This was something I had never really considered before. One part that I found particularly surprising was the issue of copyright assignment. The idea that contributors willingly give their time and, effort, but do not fully retain control over their own work, feels slightly unfair. It becomes even more concerning when that ownership shifts to a different company with a completely different approach, as happened when control moved from Sun Microsystems to Oracle.

What stood out to me the most was the community's decision to fork and, create LibreOffice. It was clearly not an easy decision, and, it came with real risks like instability and, fragmentation. At the same time, it showed a strong commitment to values such as transparency and, shared governance.

Personally, I feel that the decision to fork was justified. Staying within the existing system might have been easier in the short term, but it probably would not have solved the deeper structural problems. In a way, choosing long-term control over short-term convenience seems more meaningful. If I imagine myself in a similar situation, I think I would have made the same choice, even though it would not be easy. This case has made me realize that open source is not just about technology. It is also about trust, fairness, and, how power is shared among people involved in building something together.

Conclusion

The LibreOffice case demonstrates that open source is not only a technical framework, but also a system shaped by human relationships and, institutional structures. While licensing ensures legal freedom, it does not guarantee fairness in governance.

The events surrounding OpenOffice highlight how quickly trust can erode when control becomes centralized, even within an open ecosystem. The decision to fork was not simply a rejection of corporate authority, but an attempt to restore balance and, accountability. Ultimately, this case suggests that the long-term success of open source projects depends not just on code quality, but on how power, responsibility, and, trust are distributed among those who build and, sustain them.